

MODULE - 4 DATABASE ASSIGNMENT

1. What is RDBMS?

RDBMS stands for Relational Database Management System. It's basically the foundation for SQL and all modern database systems like MySQL, Oracle, MS SQL Server, and MS Access.

Key Points:

- Stores data in tables with rows and columns
- Tables are connected through common fields (that's why it's "relational")
- Based on the relational model by E. F. Codd

Example:

In a sales tracking system, you might have three tables:

- Customers table
- Products table
- Orders table

All connected through primary keys like CID (Customer ID), OID (Order ID), and PID (Product ID).

2. What is SQL?

SQL stands for Structured Query Language. It's a computer language used to store, manage, and retrieve data from relational databases.

Different Database Systems Use Different SQL Versions:

- MS SQL Server → T-SQL (Transact-SQL)
- Oracle → PL/SQL (Procedural Language/SQL)
- MS Access → JET SQL

SQL queries are written in two main forms:

- Action queries: INSERT, UPDATE, DELETE (to modify data)
- Select queries: SELECT (to fetch data)

3. Write SQL Commands

SQL commands are divided into four main categories:

A) DDL – Data Definition Language

- CREATE: Creates a new table, view, or database object
- ALTER: Modifies an existing table or object
- DROP: Deletes an entire table, view, or database object

B) DML – Data Manipulation Language

- INSERT: Adds new records to a table
- UPDATE: Modifies existing records
- DELETE: Removes records from a table

C) DCL – Data Control Language

- GRANT: Gives access privileges to users
- REVOKE: Removes access privileges from users

D) DQL – Data Query Language

- SELECT: Retrieves records from one or more tables

4. What is JOIN?

A JOIN is used to combine rows from two or more tables based on a related column between them.

Purpose:

- Retrieves data from multiple tables in a single query
- Establishes relationships between tables through common fields

Example:

If you have a Customers table and an Orders table, you can join them using Customer ID to see which customer placed which order.

5. Write Types of Joins

There are four main types of joins in SQL:

1) INNER JOIN

- Returns only the matching rows from both tables
- Most commonly used join type
- Also known as EQUIJOIN

2) LEFT JOIN (LEFT OUTER JOIN)

- Returns all rows from the left table
- Returns matching rows from the right table
- Shows NULL for non-matching right table rows

3) RIGHT JOIN (RIGHT OUTER JOIN)

- Returns all rows from the right table
- Returns matching rows from the left table
- Shows NULL for non-matching left table rows

4) FULL JOIN (FULL OUTER JOIN)

- Combines results of both LEFT and RIGHT joins
- Returns all rows from both tables
- Shows NULL wherever there's no match

6. How Many Constraints and Describe it self

SQL constraints are rules applied to columns to maintain data integrity and accuracy. The main constraints are:

1) PRIMARY KEY

- Uniquely identifies each record in a table
- Cannot contain NULL values
- A table can have only ONE primary key

2) FOREIGN KEY

- Links two tables together
- References the primary key of another table
- Maintains referential integrity

3) UNIQUE

- Ensures all values in a column are different
- A table can have multiple UNIQUE constraints
- Can accept NULL values (unlike PRIMARY KEY)

4) NOT NULL

- Ensures a column cannot have NULL values
- Column must always contain a value

5) CHECK

- Ensures values in a column satisfy a specific condition
- Example: age > 18

6) DEFAULT

- Provides a default value when no value is specified
- Used during INSERT operations

7. Difference between RDBMS vs DBMS

- DBMS: Stores data as files with no defined relationships
- RDBMS: Stores data in tables (rows and columns) with relationships

Relationships:

- DBMS: Does not support relationships between data
- RDBMS: Supports relationships through foreign keys and joins

Examples:

- DBMS: File systems, XML databases
- RDBMS: MySQL, Oracle, MS SQL Server, PostgreSQL, MS Access

Data Handling:

- DBMS: Suitable for small amounts of data
- RDBMS: Designed for large volumes of data with complex queries

Normalization:

- DBMS: Does not support normalization
- RDBMS: Supports normalization to reduce data redundancy

8. What is an SQL Alias?

An alias is a temporary name given to a table or column for the duration of a query. It makes queries more readable and easier to work with.

Syntax for Column Alias:

```
SELECT column_name AS alias_name FROM table_name;
```

Syntax for Table Alias:

```
SELECT t1.column1, t2.column2 FROM table1 AS t1, table2 AS t2;
```

Benefits:

- Makes queries more readable
- Useful when joining multiple tables
- Avoids confusion when columns have the same name
- Gives meaningful names to calculated columns

Example:

```
SELECT customer_name AS Name, order_date AS Date FROM customers
```

AS c; Here, "Name" and "Date" are column aliases, and "c" is a table alias.

9. Write a query to create the table in Structured Query Language

Syntax:

```
CREATE TABLE table_name
( column1 datatype,
  column2 datatype,
  column3 datatype,
  PRIMARY KEY (column_name)
);
```

Example:

```
CREATE TABLE Students (
  StudentID INT,
  FirstName  VARCHAR(50),
  LastName   VARCHAR(50),
  Age INT,
  Email
  VARCHAR(100),
  PRIMARY   KEY
  (StudentID)
);
```

This creates a "Students" table with five columns and StudentID as the primary key.

10. Write a Query to Insert Data into Table .

Syntax:

```
INSERT INTO table_name (column1, column2,
column3, ...) VALUES (value1, value2, value3, ...);
```

Example 1 - Inserting a single record:

```
INSERT INTO Students (StudentID, FirstName, LastName, Age,
Email) VALUES (1, 'Krunal', 'Patel', 22, 'krunal@example.com');
```

Example 2 - Inserting specific columns only:

```
INSERT INTO Students (StudentID, FirstName,
LastName) VALUES (2, 'Rahul', 'Shah');
```

Example 3 - Inserting multiple records:

```
INSERT INTO Students (StudentID, FirstName,
LastName, Age) VALUES
(3, 'Priya', 'Modi', 21),
(4, 'Amit', 'Desai', 23),
(5, 'Neha', 'Joshi', 20);
```

11. Write a Query to Update Data into Table with Validations.

Syntax:

```
UPDATE table_name  
SET column1 = value1, column2 =  
value2 WHERE condition;
```

Example 1 - Update a single record:

```
UPDATE Students  
SET Age = 23, Email =  
'krunal.new@example.com'  
WHERE StudentID = 1;
```

Example 2 - Update multiple records with a condition:

```
UPDATE Students  
SET Age = Age + 1  
WHERE Age < 22;
```

Example 3 - Update with multiple conditions:

```
UPDATE Students  
SET Email = 'updated@example.com'  
WHERE Age >= 21 AND LastName = 'Patel';
```

12. Write a Query to Delete Data from Table with Validations.

Syntax:

DELETE FROM table_name WHERE

condition;

Example 1 - Delete a single record:

DELETE FROM Students WHERE StudentID =

5; Example 2 - Delete records based on a

condition:

DELETE FROM Students WHERE Age < Example 3

Delete with multiple

conditions:

DELETE FROM Students WHERE Age > 25 AND Email IS NULL;

Example 4 - Delete using NOT condition:

DELETE FROM Students WHERE LastName NOT IN ('Patel', 'Shah',
'Modi');

.

13. Write a Query to Insert New Column in Existing Table

Syntax:

ALTER TABLE table_name ADD column_name datatype;

Example 1 - Add a single column:

ALTER TABLE Students ADD PhoneNumber VARCHAR(15);

Example 2 - Add column with default value:

ALTER TABLE Students ADD City VARCHAR(50) DEFAULT 'Rajkot';

Example 3 - Add column with NOT NULL constraint:

ALTER TABLE Students ADD DateOfBirth DATE NOT NULL;

Example 4 - Add multiple columns:

ALTER TABLE Students
ADD Address VARCHAR(200),
ADD PostalCode VARCHAR(10), ADD Country VARCHAR(50);

14. Write a Query to Drop Table and Database

Drop Table Syntax:

```
DROP TABLE table_name;
```

Example:

```
DROP TABLE Students;
```

Drop Database

Syntax:

```
DROP DATABASE database_name;
```

Example:

```
DROP DATABASE SchoolDB;
```

Alternative - TRUNCATE TABLE:

If you want to delete all data but keep the table structure:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE Students;
```

15. Write a Query to Find Max and Min Value from Table

Syntax for MAX:

```
SELECT MAX(column_name) FROM table_name;
```

Syntax for MIN:

```
SELECT MIN(column_name) FROM table_name;
```

Example 1 - Find maximum age:

```
SELECT MAX(Age) AS MaximumAge FROM Students;
```

Example 2 - Find minimum age:

```
SELECT MIN(Age) AS MinimumAge FROM Students;
```

Example 3 - Find both max and min in one query:

```
SELECT MAX(Age) AS MaxAge, MIN(Age) AS MinAge FROM Students;
```

Example 4 - Find max and min with a condition:

```
SELECT MAX(Age) AS OldestStudent, MIN(Age) AS YoungestStudent FROM Students  
WHERE City = 'Rajkot';
```

Example 5 - Find student with maximum age:

```
SELECT * FROM Students  
WHERE Age = (SELECT MAX(Age) FROM Students);
```


16. Create Two Tables Named Seller and Product, Apply Foreign Key in Product Table, and Fetch Data Using Different Joins.

Step 1: Create Seller Table

```
CREATE TABLE Seller (  
    SellerID INT PRIMARY KEY,  
    SellerName  
    VARCHAR(100), City  
    VARCHAR(50),  
    ContactNumber VARCHAR(15)  
);
```

Step 2: Create Product Table with Foreign Key

```
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100), Price DECIMAL(10, 2),  
    SellerID INT,  
    FOREIGN KEY (SellerID) REFERENCES Seller(SellerID)  
);
```

Step 3: Insert Sample Data into Seller Table

```
INSERT INTO Seller (SellerID, SellerName, City,  
ContactNumber) VALUES  
    (1, 'Rajesh Traders', 'Ahmedabad', '9876543210'),  
    (2, 'Patel Electronics', 'Rajkot', '9123456789'),  
    (3, 'Shah Enterprises', 'Surat', '9988776655');
```

Step 4: Insert Sample Data into Product Table

```
INSERT INTO Product (ProductID, ProductName, Price,  
SellerID) VALUES  
    (101, 'Laptop', 45000.00, 1),  
    (102, 'Mobile Phone', 25000.00, 2),  
    (103, 'Tablet', 15000.00, 1),  
    (104, 'Smart Watch', 8000.00, NULL);
```

Fetch Data Using INNER JOIN:

```
SELECT Product.ProductID, Product.ProductName, Product.Price,  
    Seller.SellerName, Seller.City  
FROM Product  
INNER JOIN Seller ON Product.SellerID =  
Seller.SellerID;
```

Fetch Data Using LEFT JOIN:

```
SELECT Product.ProductID, Product.ProductName, Product.Price,  
    Seller.SellerName, Seller.City  
FROM Product  
LEFT JOIN Seller ON Product.SellerID = Seller.SellerID;
```

Fetch Data Using RIGHT JOIN:

```
SELECT Product.ProductID, Product.ProductName, Product.Price,  
       Seller.SellerName, Seller.City  
FROM Product  
RIGHT JOIN Seller ON Product.SellerID =  
Seller.SellerID; Fetch Data Using FULL JOIN:  
SELECT Product.ProductID, Product.ProductName, Product.Price,  
       Seller.SellerName, Seller.City  
FROM Product  
FULL JOIN Seller ON Product.SellerID = Seller.SellerID;
```

17. What is API Testing?

API Testing is a type of software testing that focuses on testing Application Programming Interfaces (APIs) directly, rather than testing the user interface.

Definition:

An API (Application Programming Interface) is a software interface that allows two applications to communicate with each other without any user intervention.

API testing ensures that the API works correctly, handles errors properly, returns accurate data, and performs well under different condition

18. Types of API Testing

There are three main types of APIs based on access and usage:

1) Open APIs (Public APIs)

- Publicly available to everyone
- No access restrictions
- Example: Google OAuth APIs
- Usually require API keys for tracking usage
- Documentation is publicly available

2) Partner APIs

- Require specific rights or licenses to access
- Not available to the general public
- Shared between business partners
- Require authentication and authorization
- Access is controlled and monitored
- Used for B2B (Business-to-Business) integrations

3) Internal APIs (Private APIs)

- Developed for use within an organization only
- Not exposed to external users
- Help improve productivity of internal teams
- Improve communication between internal systems

19. What is Responsive Testing?

Responsive Testing is the process of testing a web application or website to ensure it works correctly and displays properly across different devices, screen sizes, and orientations.

Definition:

A responsive web design is a flexible design that adapts to any device - from mobile phones to tablets to desktops. It improves the user's browsing experience by automatically adjusting the layout, content, and functionality based on the device being used.

Responsive testing ensures users have a consistent and positive experience regardless of which device they use to access the website.

20. Which Types of Tools are Available for Responsive Testing?

- 1) LT Browser
 - Desktop application for testing responsive websites
 - Allows testing across multiple device viewports simultaneously
- 2) LambdaTest
 - Cloud-based cross-browser testing platform
 - Provides responsive testing on real browsers and devices
- 3) Google Resizer
 - Simple tool to visualize websites on different screen sizes
- 4) Am I Responsive
 - Web-based tool
 - Shows your website on multiple devices at once
 - URL: <https://ui.dev/amiresponsive>
 - Just enter your URL and see it across different device mockups
- 5) Pixel Tuner
 - Helps test and fine-tune responsive designs
 - Ensures pixel-perfect layouts

21. What is the Full Form of .ipa and .apk?

.APK - Android Application Package

- Stands for Android Application Package (or Android Package Kit)
- File format used by Android OS to distribute and install apps
- Similar to .exe files on Windows
- Contains all necessary components for an app to run on Android
- Developers compile Android apps into .apk format for distribution

.IPA - iOS App Store Package

- Stands for iOS App Store Package
- File format used for installing apps on Apple's iOS
- Works on iPhone, iPad, and iPod Touch
- Can only be installed on Apple devices running iOS
- Contains compiled application code and resources
- Typically distributed through the Apple App Store

22. How to create step for to open the developer option mode ON?

Step 1: Open Settings

- Go to your device's Settings application

Step 2: Navigate to About Phone

- Scroll down and find "About Phone" or "About Device"
- Tap on it to open

Step 3: Find Build Number

- Scroll down to find "Build Number" or "Build Version"
- On some devices, this may be under "Software Information"

Step 4: Tap Build Number 7 Times

- Tap on "Build Number" continuously 7 times
- You'll see a countdown message: "You are now X steps away from being a developer"
- After the 7th tap, you'll see: "You are now a developer!"

Step 5: Enable Developer Options

- Go back to the main Settings menu
- You should now see "Developer Options" (usually under "System")
- Tap on "Developer Options"

Step 6: Turn ON Developer Options

- Toggle the switch at the top to ON
- This activates all developer settings

Common Developer Options Used in Testing:

- USB Debugging - Essential for connecting to testing tools like Appium
- Stay Awake - Keeps screen on while charging
- Show Touches - Displays touch points on screen
- Pointer Location - Shows pointer/touch coordinates